ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO THỰC HÀNH

MÔN HỌC: HỆ ĐIỀU HÀNH (CS333)

LAB 1
XV6 AND UNIX UTILITIES

Sinh viên thực hiện: Võ Nhất Bảo 24120166
Phạm Lê Huy Hoàng 24120182

Mã nhóm: 24120166_24120182

Thành phố Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Thông tin nhóm và phân công công việc **2**

2 Tổng quan và môi trường thực hiện **2**

3 Bài 1 – sleep **3**

4 Bài 2 – sixfive **4**

5 Bài 3 – memdump **5**

6 Bài 4 – find **7**

7 Bài 5 – find -exec **8**

8 Kiểm thử **9**

9 Thảo luận và tự đánh giá **10**

10 Tài liệu tham khảo **10**

1. Thông tin nhóm và phân công công việc

STT	MSSV	Họ và tên	Công việc đảm nhận	Đóng góp
1	24120166	Võ Nhất Bảo	Thiết lập môi trường phát triển trên NixOS (toolchain RISC-V, QEMU, GDB); phân tích, cài đặt và kiểm thử `sleep`, `sixfive`.	50%
2	24120182	Phạm Lê Huy Hoàng	Phân tích, cài đặt và kiểm thử `memdump`, `find` và `find -exec`.	50%
Công việc chung			Chạy autograder và kiểm thử bổ sung, rà soát chéo mã nguồn, viết báo cáo, đóng gói bài nộp.	100%

Bảng 1. Phân công công việc và tỷ lệ đóng góp của các thành viên.

Bài	Chương trình	Tệp mã nguồn	Nội dung chính	Hoàn thành	Autograder
1	`sleep`	`user/sleep.c`	Dừng n tick bằng system call `pause`	100%	25/25
2	`sixfive`	`user/sixfive.c`	Lọc số chia hết cho 5 hoặc 6 từ tệp/stdin	100%	35/35
3	`memdump`	`user/memdump.c`	Diễn giải vùng nhớ theo chuỗi định dạng	100%	25/25
4	`find`	`user/find.c`	Tìm tệp/thư mục theo tên, duyệt đệ quy	100%	40/40
5	`find -exec`	`user/find.c`	Chạy lệnh cho mỗi kết quả (fork/exec/wait)	100%	30/30
–	`time.txt`	`time.txt`	Số giờ thực hiện lab của nhóm	100%	1/1
Tổng kết quả `make grade`					**156/156**

Bảng 2. Mức độ hoàn thành các yêu cầu của Lab 1.

2. Tổng quan và môi trường thực hiện

Lab 1 giúp làm quen với xv6 – hệ điều hành dạy học do MIT phát triển, mô phỏng Unix Version 6 và chạy trên RISC-V 64 bit thông qua QEMU. Nhóm cài đặt bốn chương trình user-space cho năm yêu cầu. Các chương trình chỉ dùng system call của xv6 (chủ yếu là `pause`, `open`, `read`, `close`, `fstat`, `fork`, `exec`, `wait`, `exit`) và thư viện user đi kèm (`ulib.c`, `printf.c`, `umalloc.c`); kernel và bộ chấm điểm được giữ nguyên.

2.1. Môi trường phát triển

Thành phần	Phiên bản / ghi chú
Máy host	NixOS 26.05, x86-64
Trình biên dịch chéo	`riscv64-none-elf-gcc` 15.2.0, GNU Binutils 2.46
Trình giả lập	QEMU 10.2.4 (`qemu-system-riscv64`, 3 CPU)
Gỡ lỗi và công cụ khác	GDB 17.2, GNU Make 4.4.1, Python 3.13 (chạy autograder), Git
Mã nguồn	`git://g.csail.mit.edu/xv6-labs-2026`, nhánh `util`, commit gốc `8138d07`

Bảng 3. Môi trường thực hiện.

Hướng dẫn cài đặt của môn học dành cho Ubuntu/WSL (`apt`) và macOS (Homebrew), không áp dụng trực tiếp cho NixOS. Vì vậy nhóm viết tệp `shell.nix` khai báo bộ biên dịch chéo `pkgsCross.riscv64-embedded`, QEMU, GDB, Make và Python. Tệp này đặt `TOOLPREFIX=riscv64-none-elf-` để Makefile gọi đúng trình biên dịch. Nó cũng tắt các cờ hardening mà Nix tự thêm vào (`hardeningDisable`), vì xv6 tự cung cấp runtime, linker script và các cờ biên dịch freestanding.

2.2. Quy trình thực hiện

1. **Lấy mã nguồn:** clone repo `xv6-labs-2026`. Repo tự checkout nhánh `util`; tệp `conf/lab.mk` đặt `LAB=util` để Makefile bật các chương trình và test của lab.
2. **Build và khởi động:** chạy `nix-shell`, sau đó `make qemu`. Makefile biên dịch kernel, `mkfs` và các chương trình user, tạo ảnh đĩa `fs.img`, rồi boot QEMU tới dấu nhắc của shell. Thoát QEMU bằng `Ctrl-a x`.

3. **Với mỗi bài:** viết `user/<tên>.c`, thêm `$U/_<tên>` vào `UPROGS`, chạy thử bằng `make qemu`, chấm bằng `make GRADEFLAGS=<tên> grade`, rồi commit. Mỗi chương trình được commit riêng trên nhánh `util`.
4. **Hoàn tất:** tạo `time.txt`, chạy `make grade`, kiểm tra `git status` sạch, chạy `make zipball` để tạo `lab.zip`, rồi đóng gói cùng báo cáo.

Ngoài thư mục `user/`, thay đổi duy nhất là một dòng trong khối `ifeq ($(LAB),util)` của `Makefile`:

```
ifeq ($(LAB),util)
    UEXTRA += user/findtest.sh
    UEXTRA += user/sixfive.txt
    UPROGS += $U/_sleep $U/_sixfive $U/_memdump $U/_find
endif
```

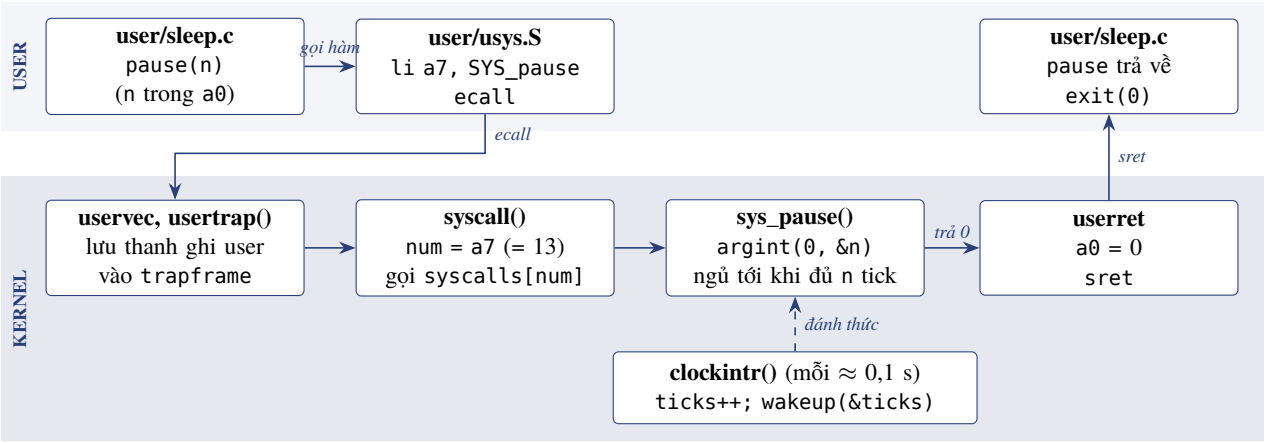
`mkfs` chỉ đưa vào `fs.img` các tệp có trong `UPROGS` và `UEXTRA`; thiếu tệp nào thì shell báo `exec <tên> failed`. Ở bản 2026, target `qemu` phụ thuộc `newfs.img`, nên mỗi lần `make qemu` ảnh đĩa được tạo lại từ đầu. Muốn giữ các tệp đã tạo ở lần chạy trước thì dùng `make qemu-fs`.

3. Bài 1 – sleep

Yêu cầu. `sleep <n>` dừng trong `n` tick bằng system call `pause`. Thiếu đối số thì in thông báo lỗi. Chương trình kết thúc bằng `exit(0)`.

3.1. Tick và đường đi của system call `pause`

Tick là đơn vị thời gian của kernel xv6, tương ứng một lần ngắt timer. `kernel/start.c` và `clockintr()` (`kernel/trap.c`) hẹn ngắt kế tiếp sau 10^6 chu kỳ của bộ đếm `time`, khoảng 0,1 giây trên QEMU. Mỗi lần ngắt, CPU 0 tăng biến toàn cục `ticks` và gọi `wakeup(&ticks)`. Đo trên máy của nhóm, `sleep 10` kéo dài khoảng 0,996 giây, khớp với ước lượng $10 \times 0{,}1$ s.



Hình 1. Đường đi của lời gọi `pause(n)` từ user mode vào kernel và quay lại.

1. `pause` khai báo trong `user/user.h`; thân hàm là stub do `user/usys.pl` sinh ra. Stub nạp `SYS_pause` = 13 vào `a7`; theo quy ước gọi hàm RISC-V, tham số `n` đã nằm trong `a0`. Sau đó stub thực thi `ecall`.
2. `ecall` chuyển CPU sang supervisor mode và nhảy tới `uservec` (`kernel/trampoline.S`), nơi thanh ghi user được lưu vào `trapframe`. `usertrap()` thấy `scause = 8` (system call) nên gọi `syscall()`. Hàm này đọc `trapframe->a7` và gọi `syscalls[13]`, tức `sys_pause()`.
3. `sys_pause()` lấy `n` bằng `argint(0, &n)`, tức đọc `trapframe->a0`. Hàm ghi nhớ `ticks0 = ticks`, rồi ngủ trên kênh `&ticks` cho tới khi `ticks - ticks0 >= n`; process bị kill thì trả về -1. Kết quả được ghi vào `trapframe->a0`, rồi `userret` và `sret` đưa CPU về user mode, ngay sau lệnh `ecall`.

3.2. Thiết kế và cài đặt

- **Kiểm tra số đối số:** nếu `argc != 2` thì in `Usage: sleep ticks` ra stderr (fd 2) rồi `exit(1)`.
- **Kiểm tra chuỗi số:** `atoi` của xv6 dừng ở ký tự đầu tiên không phải chữ số và không kiểm tra tràn số, nên `sleep -1` hay `sleep 99999999999` sẽ bị hiểu sai mà không báo lỗi. Chương trình chỉ chấp nhận chữ số. Trước mỗi lần nhân 10, nó kiểm tra `ticks > (2147483647 - d) / 10` để phát hiện tràn `int`. Chuỗi rỗng, số âm, ký tự lạ và số quá lớn đều bị từ chối với `exit(1)`.
- **Gọi pause:** `pause` trả về giá trị âm (process bị kill) thì `exit(1)`; bình thường thì `exit(0)`.

user/sleep.c – kiểm tra chuỗi số và gọi pause

```
  for (char *p = argv[1]; *p; p++) {
    if (*p < '0' || *p > '9' || ticks > (2147483647 - (*p - '0')) / 10) {
      fprintf(2, "sleep: ticks must be a non-negative integer\n");
      exit(1);
    }
```

```
    ticks = ticks * 10 + (*p - '0');
  }
  if (pause(atoi(argv[1])) < 0) {
    fprintf(2, "sleep: pause failed\n");
    exit(1);
  }
  exit(0);
```

```
$ sleep
Usage: sleep ticks
$ sleep 10
$ echo awake
awake
$ sleep 0
$ sleep -1
sleep: ticks must be a non-negative integer
$
```

Hình 2. `sleep` chạy trong xv6.

Thiếu đối số thì chương trình in hướng dẫn sử dụng. `sleep 10` trả lại dấu nhắc sau khoảng 1 giây, `sleep 0` trả về ngay, còn `sleep -1` bị từ chối. Autograder đặt breakpoint tại `sys_pause` để xác nhận chương trình thực sự gọi system call, chứ không dùng vòng lặp chờ bận.

4. Bài 2 – sixfive

Yêu cầu. Với mỗi tệp, in mọi *số* chia hết cho 5 hoặc 6 theo thứ tự xuất hiện. Số là dãy chữ số được phân cách bởi các ký tự trong `" -\r\t\n./,"`. Không có đối số thì đọc stdin; không mở được tệp thì in `sixfive: cannot open xxx`.

4.1. Phân tích định nghĩa "số"

Điểm mấu chốt: chỉ ký tự phân cách (hoặc đầu/cuối tệp) mới kết thúc một token. Dãy chữ số dính với bất kỳ ký tự nào khác thì cả token không phải là số. Bảng 4 phân tích các trường hợp lấy từ dữ liệu kiểm thử thật (`sixfive.txt` và `README` của xv6).

Đoạn input	Kết quả	Giải thích
`18-4`	`18`	`-` là phân cách; 18 chia hết cho 6, còn 4 không chia hết cho 5 hay 6.
`06` ở cuối tệp, không có `\n`	`6`	Token cuối kết thúc bởi EOF. Chương trình in giá trị số nên bỏ số 0 đứng đầu.
`Version 6 (v6).`	`6`	`6` đứng giữa hai dấu cách. `(v6)` không phải số vì chữ số dính với `v`, `(` và `).`
`.../6.1810/`	`6`, `1810`	`.` và `/` đều là phân cách.
`(June 14,\n2000))`	`14` bị loại	`14` hợp lệ nhưng không chia hết. `)` không phải phân cách nên `2000))` không hợp lệ, dù 2000 chia hết cho 5.

Bảng 4. Một số trường hợp tách token đặc trưng.

4.2. Thuật toán

Chương trình đọc input theo khối 512 byte và xử lý từng ký tự như một máy trạng thái. Trạng thái gồm `token` (các chữ số đã gom), `len`, `valid` (token còn hợp lệ hay không) và `remainder` (phần dư theo modulo 30).

Ký tự `c`	Hành động
Thuộc tập phân cách	Gọi `emit()` cho token hiện tại, sau đó đặt lại `len = 0`, `remainder = 0`, `valid = 1`.
Chữ số `'0'`–`'9'`	Nếu `valid`: nối `c` vào `token`, cập nhật `remainder = (10·remainder + d) mod 30`.
Ký tự khác (chữ cái, ngoặc, …)	Đặt `valid = 0`: token bị loại cho tới ký tự phân cách kế tiếp.
EOF (`read` trả về 0)	Gọi `emit()` lần cuối để không bỏ sót số ở cuối tệp.

Bảng 5. Quy tắc chuyển trạng thái; `emit()` in token nếu `valid`, `len > 0` và `remainder` chia hết cho 5 hoặc 6.

Chống tràn số. Chuyển token sang `int` sẽ bị tràn khi số dài hơn 10 chữ số, nên chương trình chỉ lưu phần dư theo modulo 30. Có hai tính chất bảo đảm cách này đúng:

- Vì $30 = \mathrm{BCNN}(5, 6)$, nếu $n \equiv r \pmod{30}$ thì $5 \mid n \Leftrightarrow 5 \mid r$ và $6 \mid n \Leftrightarrow 6 \mid r$.
- $(10a + d) \bmod 30 = (10\,(a \bmod 30) + d) \bmod 30$, nên phần dư được cập nhật dần theo từng chữ số mà không vượt quá 29.

Để in đúng số dài tùy ý, các chữ số được lưu trong buffer cấp phát động (khởi đầu 64 byte, nhân đôi khi đầy). Khi in, `emit()` bỏ các số 0 đứng đầu, nên `06` được in thành `6` – đúng định dạng autograder so khớp (`^6`).

Các chi tiết khác.

- **Số vắt qua hai khối đọc:** các biến trạng thái nằm ngoài vòng `read`, nên số bị cắt ở biên khối 512 byte vẫn được ghép đúng.
- **Token cuối tệp:** khi gặp EOF, chương trình gọi `emit()` thêm một lần (dòng 64) cho token cuối.
- **Ký tự NUL:** `'\0'` không phải phân cách. Với `strchr` chuẩn của C, `strchr(s, '\0')` trả về vị trí ký tự kết thúc chuỗi, nên chương trình kiểm tra `c != '\0'` trước khi gọi `strchr`.

- **Nhiều tệp và xử lý lỗi:** trạng thái là biến cục bộ của `sixfive(fd)`, nên mỗi tệp được xử lý độc lập; mọi fd đã mở đều được `close`. Tệp mở lỗi thì in thông báo, ghi nhận lỗi rồi xử lý tiếp tệp sau; mã thoát là 1 nếu có lỗi.
- **Độ phức tạp:** $O(N)$ thời gian với N byte input; bộ nhớ $O(L + 512)$ với L là độ dài token hợp lệ dài nhất.

user/sixfive.c – vòng xử lý chính (lược các dòng 41–54 mở rộng buffer)

```
30   while ((n = read(fd, input, sizeof(input))) > 0) {
31     for (int i = 0; i < n; i++) {
32       char c = input[i];
33       // NUL is not a separator, even though strchr can find it at the end.
34       if (c != '\0' && strchr(" -\r\t\n./,", c)) {
35         emit(token, len, valid, remainder);
36         len = remainder = 0;
37         valid = 1;
38       } else if (c < '0' || c > '9') {
39         valid = 0;
40       } else if (valid) {
55         token[len++] = c;
56         // Modulo 30 preserves divisibility by 5 and 6 without overflow.
57         remainder = (remainder * 10 + c - '0') % 30;
```

```
$ sixfive sixfive.txt
5
100
18
6
$ echo xv6 6x 12 30 7 | sixfive
12
30
$ sixfive absent
sixfive: cannot open absent
$
```

Với `sixfive.txt`, chương trình in 5, 100, 18, 6. Với stdin `xv6 6x 12 30 7`, hai token `xv6` và `6x` bị loại vì chữ số dính chữ cái, 7 không chia hết, nên chỉ 12 và 30 được in. Với tệp không tồn tại, chương trình in đúng thông báo `sixfive: cannot open absent`.

Hình 3. `sixfive` với tệp, stdin và tệp không tồn tại.

5. Bài 3 – memdump

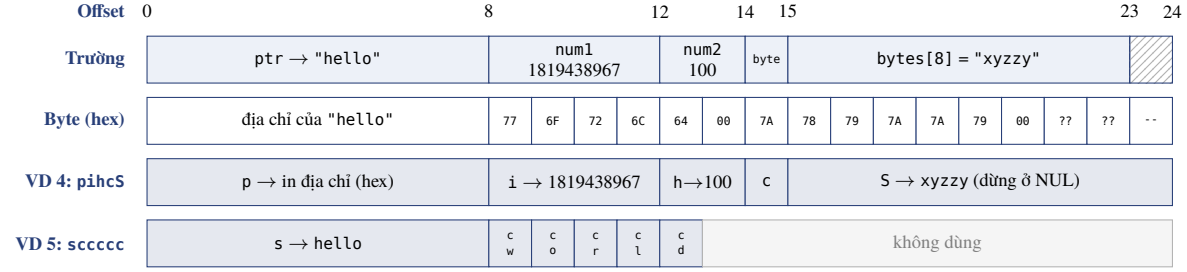
Yêu cầu. Cài đặt `memdump(char *fmt, char *data, int len)`: diễn giải `len` byte tại `data` theo chuỗi định dạng `fmt`; mỗi ký tự định dạng in một giá trị trên một dòng. Hàm `main` do đề cung cấp: không có đối số thì chạy năm ví dụ dựng sẵn; có một đối số thì đọc tối đa 512 byte từ stdin làm dữ liệu.

Ký tự	Số byte	Kiểu C	In bằng	Ý nghĩa
`i`	4	`int`	`%d`	Số nguyên có dấu 32 bit
`p`	8	`uint64`	`%lx`	Số 64 bit dạng hex, không có tiền tố `0x`
`h`	2	`short`	`%d`	Số nguyên có dấu 16 bit
`c`	1	`char`	`%c`	Một ký tự ASCII
`s`	8	`char *`	`%s`	8 byte là *địa chỉ* của một C string; in chuỗi tại địa chỉ đó
`S`	phần còn lại	`char[]`	từng ký tự	Các byte *chính là* nội dung chuỗi; in tới NUL hoặc hết dữ liệu

Bảng 6. Các ký tự định dạng của `memdump`.

5.1. Bố cục bộ nhớ của các ví dụ

RISC-V lưu số nguyên theo thứ tự *little-endian*: byte ở địa chỉ thấp là byte thấp của giá trị. Hình 4 vẽ `struct sss` của ví dụ 4 và 5. Các trường chiếm 23 byte; trình biên dịch thêm 1 byte đệm để kích thước là bội của 8 (độ căn chỉnh của con trỏ `ptr`), nên `sizeof(struct sss) = 24`.



Hình 4. Bố cục `struct sss` (24 byte) và cách ví dụ 4, 5 diễn giải cùng một vùng nhớ.

- **Ví dụ 4 (`pihcS`):** `p` in địa chỉ chuỗi `"hello"` dạng hex (phụ thuộc bản build; ở máy nhóm là `CD0`). `i` đọc `77 6F 72 6C`, tạo thành `0x6C726F77` = 1819438967. `h` đọc `64 00`, ra 100. `c` đọc `7A`, ra `z`. `S` in phần còn lại tới NUL, ra `xyzzy`.
- **Ví dụ 5 (`sccccc`):** `s` đọc `ptr` và in `hello`. Năm lệnh `c` đọc offset 8–12: bốn byte của `num1` (`w o r l`) và byte thấp của `num2` (`0x64` = `d`), ghép thành `world`.
- **Ví dụ 1–3:** `ii` đọc hai phần tử của `{61810, 2026}`. `S` đọc trực tiếp các ký tự của `"a string"`. `s` đọc 8 byte của *biến con trỏ* `s` rồi in chuỗi `"another"` mà nó trỏ tới. Đây là khác biệt cốt lõi: `s` truy cập gián tiếp qua con trỏ, còn với `S` thì dữ liệu chính là chuỗi.
- **Chế độ stdin:** `echo deadc0de | memdump hhcccc` nhận 9 byte `64 65 61 64 63 30 64 65 0A`. `h` thứ nhất đọc `64 65`, ra `0x6564` = 25956; `h` thứ hai đọc `61 64`, ra `0x6461` = 25697; bốn lệnh `c` in `c`, `0`, `d`, `e`.

```
$ memdump
Example 1:
61810
2026
Example 2:
a string
Example 3:
another
Example 4:
CD0
1819438967
100
z
xyzzy
Example 5:
hello
w
o
r
l
d
$ echo deadc0de | memdump hhcccc
25956
25697
c
0
d
e
$ echo a | memdump i
memdump: not enough data for 'i'
$
```

Hình 5. Năm ví dụ, chế độ stdin và lỗi thiếu dữ liệu.

5.2. Thuật toán và cài đặt

Với mỗi ký tự của `fmt`, chương trình làm bốn bước:

1. Xác định số byte `size`; ký tự định dạng lạ thì báo lỗi và dừng.
2. Nếu `len < size` thì in `memdump: not enough data for 'x'` và dừng.
3. Sao chép `size` byte vào biến đúng kiểu bằng `memmove`, rồi in giá trị.
4. Tăng `data` và giảm `len` đi `size`.

Riêng `S` dùng hết phần dữ liệu còn lại nên kết thúc vòng lặp. Ba quyết định thiết kế quan trọng:

- **Kiểm tra biên trước khi đọc.** `main` đã `memset` buffer về 0, nên đọc lố `len` không gây lỗi bộ nhớ nhưng cho kết quả sai. Ví dụ với `echo a | memdump i`, nếu không kiểm tra thì chương trình đọc 4 byte `61 0A 00 00` và in 2657 thay vì báo thiếu dữ liệu; autograder kiểm tra đúng trường hợp này. Dữ liệu từ stdin cũng có thể không có NUL, nên `S` dừng ở NUL *hoặc* khi hết `len`.
- **`memmove` thay vì ép kiểu con trỏ.** Với định dạng như `ci`, trường `i` bắt đầu ở offset 1, không chia hết cho 4. Khi đó `*(int *)data` là truy cập không căn chỉnh, một hành vi không xác định theo chuẩn C; `memmove` sao chép từng byte nên đúng với mọi offset.
- **Đặc tả `printf` cho `p`.** Trong `user/printf.c`, `%x` lấy tham số kiểu `uint32` nên mất 32 bit cao, còn `%p` luôn in tiền tố `0x` và đủ 16 chữ số. Chỉ `%lx` in đúng giá trị 64 bit, ví dụ `6867666564636261` cho input `abcdefgh`.

user/memdump.c – xác định kích thước, kiểm tra biên và xử lý S

```
63   for (; *fmt; fmt++) {
64     int size;
65     switch (*fmt) {
66     case 'i': size = 4; break;
67     case 'p': case 's': size = 8; break;
68     case 'h': size = 2; break;
69     case 'c': size = 1; break;
70     case 'S': size = 0; break;
71     default:
72       fprintf(2, "memdump: unknown format '%c'\n", *fmt);
73       return;
74     }
75     if (len < size) {
76       fprintf(2, "memdump: not enough data for '%c'\n", *fmt);
77       return;
78     }
```

```
106    case 'S':
107      // Stdin need not include a NUL: never scan beyond the valid bytes.
108      for (int i = 0; i < len && data[i] != '\0'; i++)
109        printf("%c", (uchar)data[i]);
110      printf("\n");
111      return; // S consumes the rest of the data.
```

6. Bài 4 – find

Yêu cầu. `find <path> <name>` tìm mọi tệp *và thư mục* có tên đúng bằng `name` trong cây thư mục gốc `path`, rồi in đường dẫn của chúng.

6.1. Cấu trúc thư mục trong xv6

Thư mục là một tệp đặc biệt (`st.type == T_DIR`). Nội dung của nó là mảng `struct dirent` 16 byte (`kernel/fs.h`): 2 byte `inum` và 14 byte tên (`DIRSIZ = 14`). Tên dài đúng 14 ký tự thì *không có* NUL kết thúc. Entry có `inum = 0` là ô trống (tệp đã xóa). Mọi thư mục đều có hai entry đặc biệt: `.` trỏ tới chính nó và `..` trỏ tới thư mục cha. Để duyệt thư mục, chương trình `open` nó, dùng `fstat` để phân biệt tệp với thư mục, rồi `read` các `struct dirent`.

6.2. Thuật toán duyệt theo chiều sâu (DFS)

```
find(path):
  fd = open(path); fstat(fd, &st)
  nếu st.type != T_DIR:
    close(fd); match(path); return
  đọc toàn bộ dirent; close(fd)
  match(path)   # thư mục cũng có thể khớp
  với mỗi entry e:
    bỏ qua nếu e.inum == 0
         hoặc e.name là "." / ".."
    find(path + "/" + e.name)

match(path):
  in path nếu tên cuối == name
```

```
find(".")
├─ find("./README")
├─ find("./b")        → ./b
└─ find("./a")
   ├─ find("./a/b")   → ./a/b
   └─ find("./a/aa")
      └─ find("./a/aa/b")
                      → ./a/aa/b
```

Cây lời gọi đệ quy cho lệnh `find . b` trong ví dụ của đề. Thứ tự kết quả là thứ tự entry trong thư mục, chương trình không sắp xếp.

6.3. Các vấn đề cần xử lý

- **Bỏ qua `.` và `..`:** đệ quy vào `.` tạo đường dẫn `./././…` cho tới khi vượt `MAXPATH`; đệ quy vào `..` quay lên thư mục cha và duyệt lại cả cây, làm mỗi kết quả bị in nhiều lần. Autograder kiểm tra mỗi tệp chỉ được in đúng một lần.
- **Tên dài 14 ký tự:** `de.name` được chép vào `name[DIRSIZ + 1]` và gán `name[DIRSIZ] = '\0'` trước khi so sánh bằng `strcmp`. Toán tử == chỉ so sánh địa chỉ của hai chuỗi.
- **Giới hạn file descriptor:** mỗi process có tối đa `NOFILE = 16` fd, trong đó fd 0, 1, 2 đã dùng cho stdin, stdout, stderr. Nếu giữ fd thư mục mở trong lúc đệ quy (như cách `ls.c` đọc từng entry), mỗi tầng chiếm một fd, và cây sâu hơn khoảng 13 tầng sẽ khiến `open` thất bại. Vì vậy nhóm đọc toàn bộ thư mục một lần vào buffer cấp phát động kích thước `st.size`, kiểm tra số byte đọc được, rồi `close(fd)` *trước* khi đệ quy. Kiểm thử bổ sung xác nhận chương trình duyệt đúng cây sâu 24 tầng.
- **Đường dẫn:** đường dẫn con được ghép trong buffer cấp phát động để stack frame nhỏ, vì user stack của lab util chỉ có 2 trang (8 KB). Đường dẫn vượt `MAXPATH = 128` bị từ chối. Dấu `/` chỉ được chèn khi `path` chưa kết thúc bằng `/`; `main` bỏ các `/` thừa ở cuối đối số (`a/` thành `a`, còn `/` giữ nguyên). Nhờ vậy kết quả không có dạng `a//b`.
- **Khớp tên và lỗi:** `match()` lấy phần sau dấu `/` cuối cùng và so với `name`, áp dụng cho cả tệp lẫn thư mục (ví dụ `find . aa` in `./a/aa`). Đường dẫn không mở, `fstat` hay đọc được thì chương trình in lỗi ra stderr, bỏ qua nhánh đó và tiếp tục duyệt các nhánh khác; mã thoát là 1.

user/find.c – đọc thư mục và duyệt entry

```
67    struct dirent *entries = malloc(st.size);
68    if (entries == 0) {
69      fprintf(2, "find: out of memory\n");
70      close(fd);
71      return 1;
72    }
73    int n = read(fd, entries, st.size);
74    close(fd);
75    if (n < 0 || (uint)n != st.size || n % sizeof(*entries) != 0) {
76      fprintf(2, "find: cannot read directory %s\n", path);
77      free(entries);
78      return 1;
79    }
```

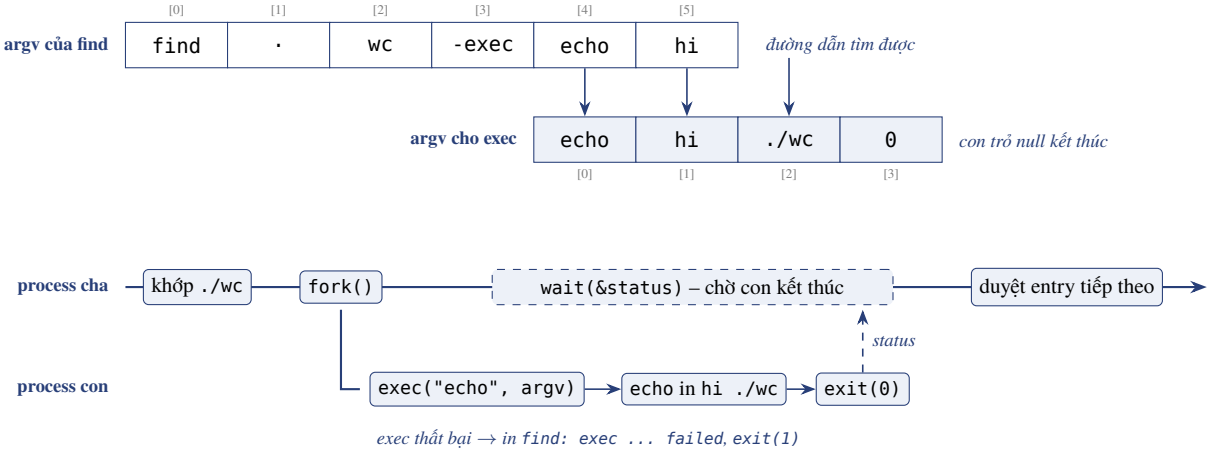
```
84    for (int i = 0; i < n / sizeof(*entries); i++) {
85      if (entries[i].inum == 0)
86        continue;
87      char name[DIRSIZ + 1];
88      memmove(name, entries[i].name, DIRSIZ);
89      name[DIRSIZ] = '\0';
90      if (strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
91        continue;
```

```
$ echo > b
$ mkdir a
$ echo > a/b
$ mkdir a/aa
$ echo > a/aa/b
$ find . b
./b
./a/b
./a/aa/b
$ find . aa
./a/aa
$
```

Hình 6. Ba tệp `b` ở các độ sâu khác nhau và thư mục `aa` khớp tên.

7. Bài 5 – find -exec

Yêu cầu. Với `find <path> <name> -exec cmd [args...]`, chương trình chạy `cmd args... <đường-dẫn>` cho mỗi kết quả, thay vì in đường dẫn. Không có `-exec` thì `find` giữ nguyên hành vi của bài 4.



Hình 7. Dựng vector đối số cho `find . wc -exec echo hi` và trình tự `fork–exec–wait`.

- **Dựng vector đối số:** các từ sau `-exec` được chép vào mảng toàn cục `command[MAXARG]`. Với mỗi kết quả, đường dẫn được đặt vào `command[ncommand]`, rồi gán `command[ncommand + 1] = 0`, vì `exec` yêu cầu `argv` kết thúc bằng con trỏ null. `main` kiểm tra `ncommand + 2 <= MAXARG` (= 32), giới hạn mà kernel áp cho `exec`. Thiếu `cmd` (ví dụ `find . b -exec`) hoặc từ thứ tư không phải `-exec` thì báo sai cú pháp.
- **`fork()`** nhân bản process `find`: trả về 0 trong process con, PID của con trong process cha, và giá trị âm nếu lỗi. Không thể gọi `exec` trực tiếp trong `find`, vì `exec` thay thế chính process đang chạy và việc duyệt cây sẽ dừng ngay ở kết quả đầu tiên.
- **Process con gọi `exec(command[0], command)`.** Thành công thì `exec` thay toàn bộ ảnh bộ nhớ của con bằng chương trình `cmd` và không bao giờ trả về. Thất bại (ví dụ lệnh không tồn tại) thì con in `find: exec <cmd> failed` và `exit(1)` ngay. Nếu không thoát, con – vốn là bản sao của `find` – sẽ tiếp tục duyệt cây, in trùng kết quả và `fork` thêm process.
- **Process cha gọi `wait(&status)`** trước khi xử lý kết quả tiếp theo. Nhờ vậy output của các lệnh không xen kẽ, process con được thu hồi (không thành zombie), và mỗi lúc chỉ có tối đa một process con. Mã thoát khác 0 của lệnh được tính vào mã thoát của `find`.
- **fd và PATH:** fd thư mục đã đóng trước khi đệ quy (mục 6.3), nên process con không thừa hưởng fd duyệt cây. Shell xv6 không có PATH: `exec("echo", …)` tìm tệp `echo` theo thư mục hiện hành, giống `user/sh.c`. Vì vậy khi đang ở thư mục con, cần ghi đường dẫn tuyệt đối, ví dụ `-exec /echo`.

user/find.c – chạy lệnh cho một kết quả

```
26    command[ncommand] = path;
27    command[ncommand + 1] = 0;
28    int pid = fork();
29    if (pid < 0) {
30      fprintf(2, "find: fork failed\n");
31      return 1;
32    }
33    if (pid == 0) {
34      exec(command[0], command);
35      fprintf(2, "find: exec %s failed\n", command[0]);
36      exit(1); // A failed exec must never resume the parent's traversal.
37    }
38    int status;
39    if (wait(&status) < 0) {
40      fprintf(2, "find: wait failed\n");
41      return 1;
42    }
43    return status != 0;
```

```
$ find . wc -exec echo hi
hi ./wc
$ find . wc -exec absent
find: exec absent failed
$ sh < findtest.sh
$ echo SCRIPT_DONE
$ $ $ $ $ hello
hello
hello
$ $ SCRIPT_DONE
$
```

Hình 8. Truyền thêm đối số, lệnh không tồn tại và script `findtest.sh`.

`findtest.sh` tạo ba tệp `a/b`, `c/b`, `b` chứa chữ `hello`, rồi chạy `find . b -exec grep hello`. Lệnh này tương đương `grep hello` lần lượt trên `./a/b`, `./c/b`, `./b`, nên in `hello` đúng ba lần. Với `find . wc -exec absent`, lỗi chỉ được báo *một* lần cho mỗi kết quả.

8. Kiểm thử

8.1. Autograder

```
$ make grade

make clean
/nix/store/hm3031qdg9p7rk6carbrk16xmhbl8vdp-riscv64-none-elf-binutils-2.46/bin/riscv64-none-elf-ld.bfd: warning:
 kernel/kernel has a LOAD segment with RWX permissions
== Test sleep, no arguments ==
$ make qemu-gdb
sleep, no arguments: OK (15.7s)
== Test sleep, returns ==
$ make qemu-gdb
sleep, returns: OK (0.6s)
== Test sleep, makes syscall ==
$ make qemu-gdb
sleep, makes syscall: OK (0.9s)
== Test sixfive test ==
$ make qemu-gdb
sixfive test: OK (1.0s)
== Test sixfive_readme ==
$ make qemu-gdb
sixfive_readme: OK (0.9s)
== Test sixfive_all ==
$ make qemu-gdb
sixfive_all: OK (1.0s)
== Test memdump, examples ==
$ make qemu-gdb
memdump, examples: OK (1.0s)
== Test memdump, format ii, S, p ==
$ make qemu-gdb
memdump, format ii, S, p: OK (1.1s)
== Test find, in current directory ==
$ make qemu-gdb
find, in current directory: OK (1.0s)
== Test find, in sub-directory ==
$ make qemu-gdb
find, in sub-directory: OK (1.2s)
== Test find, recursive ==
$ make qemu-gdb
find, recursive: OK (1.1s)
== Test exec ==
$ make qemu-gdb
exec: OK (0.8s)
== Test exec, multiple args ==
$ make qemu-gdb
exec, multiple args: OK (1.0s)
== Test exec via sh script ==
$ make qemu-gdb
exec via sh script: OK (1.4s)
== Test sleep, zero and non-numeric args return ==
$ make qemu-gdb
sleep, zero and non-numeric args return: OK (0.6s)
== Test sixfive, stdin and missing file ==
$ make qemu-gdb
sixfive, stdin and missing file: OK (0.8s)
== Test memdump, formats h c, bounds checking, usage ==
$ make qemu-gdb
memdump, formats h c, bounds checking, usage: OK (0.9s)
== Test find, missing target, no duplicates, directory match ==
$ make qemu-gdb
find, missing target, no duplicates, directory match: OK (1.5s)
== Test time ==
time: OK
Score: 156/156

$ echo $?
0
```

Hình 9. Toàn bộ output của `make grade`.

Test của autograder	Điểm
sleep	
no arguments; returns; makes syscall (breakpoint tại `sys_pause`); zero and non-numeric args	25/25
sixfive	
sixfive_test; sixfive_readme; sixfive_all; stdin and missing file	35/35
memdump	
examples; format ii, S, p; formats h c, bounds checking, usage	25/25
find	
current directory; sub-directory; recursive; missing target, no duplicates, directory match	40/40
find -exec	
exec; exec, multiple args; exec via sh script	30/30
time (`time.txt` tồn tại)	1/1
Tổng	**156/156**

Bảng 7. Các test của autograder theo bài.

Kết quả `make grade` cho toàn bộ lab: 19/19 test đạt, tổng **156/156** điểm, mã thoát 0 (Hình 9). Hai tệp `grade-lab-util` và `gradelib.py` được giữ nguyên: `git diff origin/util` chỉ gồm `Makefile`, `time.txt` và bốn tệp trong `user/`. Mã của nhóm biên dịch với `-Wall -Werror` không có cảnh báo; cảnh báo `RWX` của linker trong ảnh thuộc về kernel gốc.
Test *exec, multiple args* chạy `grep hello ./findtest.sh`: `grep` in 4 dòng chứa `hello`, cộng thêm dòng lệnh vừa gõ được console hiển thị lại, nên có đúng 5 lần `hello` như autograder đếm.

8.2. Kiểm thử bổ sung

Ngoài autograder, nhóm viết script `scripts/check_extra.py` chạy 39 ca kiểm thử trực tiếp trong QEMU. Script dùng một ảnh đĩa tạm riêng, không ảnh hưởng tới `fs.img` và bộ chấm gốc. Kết quả: **39/39** ca đạt.

Chương trình	Số ca	Nội dung kiểm tra
`sleep`	8	Thiếu/thừa đối số; `0`; `-1`; `z`; `2147483648`; số 21 chữ số; `sleep 10` trả về sau ≈ 0,996 s.
`sixfive`	6	Đủ 8 ký tự phân cách và EOF không xuống dòng; NUL không là phân cách; số vắt qua biên 512 byte và số 601 chữ số; tiếp tục sau tệp lỗi; stdin; tệp rỗng.
`memdump`	11	Struct packed với số âm và hex đủ 64 bit; `S` dừng ở NUL; `S` 512 byte không có NUL; input rỗng với `i`, `h`, `c`, `p`, `s`, `S`; dừng ở trường thứ hai thiếu dữ liệu; định dạng lạ.
`find, -exec`	14	Đúng thứ tự kết quả; khớp chính thư mục gốc; dấu `/` ở cuối; `path` là tệp; tên/đường dẫn không tồn tại; tên đủ 14 ký tự; `-exec` nối đúng đường dẫn; exec lỗi chỉ báo một lần mỗi kết quả; `-exec` thiếu lệnh; cây sâu 24 tầng không rò rỉ fd; `-exec` trong cây sâu; từ chối đường dẫn vượt `MAXPATH`.

Bảng 8. Các nhóm ca kiểm thử bổ sung.

9. Thảo luận và tự đánh giá

9.1. Khó khăn và cách khắc phục

Vấn đề	Nguyên nhân	Cách khắc phục
Không cài được công cụ theo hướng dẫn `apt`	Máy phát triển dùng NixOS	Viết `shell.nix` khai báo bộ biên dịch `riscv64-none-elf`, QEMU, GDB; đặt `TOOLPREFIX`; tắt hardening của Nix.
Tệp tạo trong xv6 biến mất ở lần chạy sau	`make qemu` bản 2026 tạo lại `fs.img` mỗi lần	Dùng `make qemu-fs` khi cần giữ dữ liệu; các kiểm thử tự tạo dữ liệu đầu vào.
`sixfive`: `06` phải in thành `6`; `2000))` không được tính	Autograder so khớp đầu dòng; token phải gồm toàn chữ số	`emit()` bỏ số 0 đứng đầu; cờ `valid` chỉ được đặt lại tại ký tự phân cách.
`memdump` in 2657 cho `echo a | memdump i`; giá trị `p` mất 32 bit cao	Buffer của `main` đã `memset` nên đọc lố không gây lỗi; `%x` của xv6 nhận `uint32`	Kiểm tra `len < size` trước mọi lần đọc; in `p` bằng `%lx`.
`find` có nguy cơ lỗi trên cây sâu	Mỗi tầng đệ quy giữ một fd; giới hạn `NOFILE = 16`	Đọc trọn thư mục rồi đóng fd trước khi đệ quy.
Script kiểm thử bổ sung báo `exec mkdir failed` sau khi `cd` vào thư mục con	Shell xv6 không tìm lệnh theo PATH	Gọi lệnh bằng đường dẫn tuyệt đối (`/mkdir`, `/echo`), thêm bước kiểm tra dữ liệu chuẩn bị rồi chạy lại toàn bộ kiểm thử.

Bảng 9. Các khó khăn gặp phải trong quá trình thực hiện.

9.2. Hạn chế

- `memdump` ở chế độ stdin chỉ đọc tối đa 512 byte (giới hạn của `main` do đề cung cấp). Định dạng `s` coi 8 byte dữ liệu là một địa chỉ hợp lệ trong process; nếu dữ liệu nhập từ stdin bị diễn giải bằng `s`, địa chỉ không hợp lệ gây page fault và kernel kết thúc process.
- `find` không sắp xếp kết quả. Mỗi thư mục được chụp lại tại thời điểm đọc, nên nếu lệnh trong `-exec` thay đổi cây thư mục trong lúc duyệt thì kết quả có thể thay đổi theo. Đường dẫn tối đa `MAXPATH = 128` byte; lệnh `-exec` tối đa `MAXARG`$-2 = 30$ từ.

9.3. Tự đánh giá theo thang điểm

Hạng mục	Điểm tối đa	Mức tự đánh giá	Minh chứng
Bài 1 – sleep	1.0	Xuất sắc	4/4 test; kiểm tra đối số, mã thoát `exit(0)`/`exit(1)`.
Bài 2 – sixfive	1.5	Xuất sắc	4/4 test; đủ ký tự phân cách, token cuối, chữ số dính chữ cái, stdin, lỗi mở tệp.
Bài 3 – memdump	2.5	Xuất sắc	3/3 test; đủ 6 định dạng, kiểm tra biên, chế độ stdin.
Bài 4 – find	2.0	Xuất sắc	4/4 test; đệ quy, bỏ `./..`, đóng fd, kiểm tra `MAXPATH`.
Bài 5 – find -exec	1.5	Xuất sắc	3/3 test; `fork`/`exec`/`wait`, con thoát khi exec lỗi.
Chất lượng mã	0.25	Xuất sắc	Hàm phụ trợ, tên biến rõ ràng, có chú thích, không cảnh báo biên dịch.
Git và `time.txt`	0.25	Xuất sắc	Nhánh `util` có commit của cả hai thành viên, mỗi chương trình một commit theo phân công; `time.txt` và báo cáo đã commit.
Báo cáo	1.0	Xuất sắc	Đủ các mục: bìa, phân công, từng bài, kiểm thử, thảo luận, tài liệu.

Bảng 10. Tự đánh giá theo rubric của đề.

10. Tài liệu tham khảo

[1] CS333 – Hệ điều hành, *Lab 1: Xv6 and Unix Utilities* (đề bài, yêu cầu nộp bài và thang điểm), 2026.

[2] MIT 6.1810, *Lab: Xv6 and Unix utilities*, Fall 2026. https://pdos.csail.mit.edu/6.828/2026/labs/util.html

[3] R. Cox, M. F. Kaashoek, R. Morris, *xv6: a simple, Unix-like teaching operating system (RISC-V)*, chương 1 "Operating system interfaces". https://mit-pdos.github.io/xv6-riscv-book/

[4] Mã nguồn xv6-labs-2026, nhánh `util`: `user/ls.c`, `user/ulib.c`, `user/printf.c`, `user/sh.c`, `kernel/sysproc.c`, `kernel/syscall.c`, `kernel/trap.c`, `kernel/fs.h`, `kernel/param.h`.

[5] Tài liệu môn học: *Seminar XV6* (kiến trúc xv6, chế độ đặc quyền, system call) và *Setup-lab* (cài đặt công cụ).

[6] Công cụ AI (OpenAI Codex, Claude): hỗ trợ thiết lập môi trường NixOS, hỗ trợ viết và rà soát mã nguồn, xây dựng script kiểm thử bổ sung, giải thích khái niệm và biên tập báo cáo. Nhóm chịu trách nhiệm về toàn bộ nội dung bài nộp.